Wentworth Institute of Technology
COMP3450 – Parallel Programming
Spring 2026

Wentworth
Computing & Data Science

Lab Assignment 1

What:

In this assignment you are to write a C program that preforms a matrix-matrix product:

1. Login to the server via ssh.
2. Copy the folder named **`PA1`** into your home directory. It is located in **`/assignments`**
3. There are two files, **`mm-student.c`**, with "starter" code for you to build on, and **`mm-student.h`**, which contains three required functions for you to implement in your code.
4. When finished, compile the code according to what we covered in lecture. **You must implement prototypes methods that I provide in the header file.**
5. Even though this is not a parallel code (we will do that next week) create a submission script and submit to the cluster. Only request one node.
6. Build your program as I have shown in class using gcc.
7. **Write you code in a way that you can compute any size square matrix-matrix product**. Below is an example of a matrix-matrix product that you might see in a math class.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix} = \begin{bmatrix} 14 & 32 & 50 \\ 32 & 77 & 122 \\ 50 & 122 & 194 \end{bmatrix}$$

Why:

When first leaning C/C++, pointers are the hardest part to deal with, especially if you have not used them before. This assignment forces you to use pointers to complete the problem. Most of the MPI functions that we will use in the future require pointers to data structures (arrays specifically) to run properly, so learning how to create and pass points to functions is imperative.

I also give you several prototype functions that you must implement. This is similar to your CS1 or CS2 days. Specifically, the function that performs the matrix-matrix product takes several **`double*`** parameters (a pointer to a **`double`**). These represent arrays in the context of our program. An important aspect of using MPI is understanding how memory is laid out and how we need to structure our data to conform to what MPI expects. In this case, you need to use a 1D array to represent the matrix. This concept is extremely useful for future assignments.

You should also think about how you might perform this calculation in parallel. Think about using both a shared memory system and a distributed memory system and how their implementation might differ. There is no wrong answer to these questions because we have not covered the topics yet, but I do want you to begin to think about your code from a parallel perspective.

How:

PBS Script
See the Lecture 2b slides for more detail. In addition, use the outline from the lectures to create a pbs script that can be used to submit your code to the cluster.

Code Setup

Use what you have learned in class to allocate memory for three arrays, that are **rows*cols** in size, representing all three matrices. The result array will be allocated in the **mm** function, while the other two matrices should be allocated in **main**. Accessing the elements of the matrix with a 2D index, e.g. a set of nested for loops (over **i** and **j**), is relatively simple:

```
int index=i*cols+j;
 matrix[index]=100;
```

Here, **cols** is the number of columns in the matrix and assumes that **i** is looping over rows and **j** is looping over columns. You are simply converting the **i,j** 2D index to a 1D index that can be used with the array that you allocated. There is also an equation to go from one index back to a 2D index that you may find useful as well.

In addition, the included header file has three prototype functions that you must implement and use in your code, **mm**, **assignA** and **assignB**. **Do not put source code in the header file**, implement the functions in your source file. **mm** performs the matrix-matrix product and **assignA/B** will fill a matrices with appropriate values for your calculations. Do not fill the matrices manually using the command line or standard in. In the future, we will use this code as a base for parallelizing a slightly easier matrix-vector product for very large matrices.

Matrix-Matrix Product
Remember that a matrix-matrix product is just a series of dot products.

1. Write a dot product and ensure that it works properly.
2. Add an outer loop around the dot product that loops over the matrix rows.
3. Add another outer loop that loops over the matrix columns.
4. Perform dot products for each row-column combination and place the result in the correct location in the result matrix.

Expected Results:

The only deliverables for this assignment are the code itself, which you will leave on the server and the output of a small test case uploaded to Brightspace, similar to what you see below. **However, make sure that your code can compute any size square matrix-vector product.**

```
1.00 2.00 3.00     1.00 4.00 7.00    14.00  32.00  50.00
4.00 5.00 6.00  *  2.00 5.00 8.00  = 32.00  77.00 122.00
7.00 8.00 9.00     3.00 6.00 9.00    50.00 122.00 194.00
```

For large cases, do not write the result to the console, I will examine your code and confirm that you get the correct result of larger matrix-matrix products.